

DECISION MAKING AND LOOPING

6.1 INTRODUCTION

We have seen in the previous chapter that it is possible to execute a segment of a program repeatedly by introducing a counter and later testing it using the if statement. While this method is quite satisfactory for all practical purposes, we need to initialize and increment a counter and test its value at an appropriate place in the program for the completion of the loop. For example, suppose we want to calculate the sum of squares of all integers between 1 and 10. We can write a program using the if statement as follows:

```

-----
sum = 0;
n = 1;
-----
L: loop:
  sum = sum + n*n;
  goto print;
  if (n == 10)
  else
  {
    n = n+1;
    goto loop;
  }
print:
-----
n = 10,
end of loop
-----

```

This program does the following things:

1. Initializes the variable **n**.
2. Computes the square of **n** and adds it to **sum**.
3. Tests the value of **n** to see whether it is equal to 10 or not. If it is equal to 10, then the program prints the results.
4. If **n** is less than 10, then it is incremented by one and the control goes back to compute the **sum** again.

The program evaluates the statement

$$\text{sum} = \text{sum} + n * n;$$

10 times. That is, the loop is executed 10 times. This number can be decreased or increased easily by modifying the relational expression appropriately in the statement if ($n == 10$). On such occasions where the exact number of repetitions are known, there are more convenient methods of looping in C. These looping capabilities enable us to develop concise programs containing repetitive processes without the use of **goto** statements.

In looping, a sequence of statements are executed until some conditions for termination of the loop are satisfied. A *program loop* therefore consists of two segments, one known as the *body of the loop* and the other known as the *control statement*. The control statement tests certain conditions and then directs the repeated execution of the statements contained in the body of the loop.

Depending on the position of the control statement in the loop, a control structure may be classified either as the *entry-controlled loop* or as the *exit-controlled loop*. The flowcharts in Fig. 6.1 illustrate these structures. In the entry-controlled loop, the control conditions are tested before the start of the loop execution. If the conditions are not satisfied, then the body of the loop will not be executed. In the case of an exit-controlled loop, the test is performed at the end of the body of the loop and therefore the body is executed unconditionally for the first time.

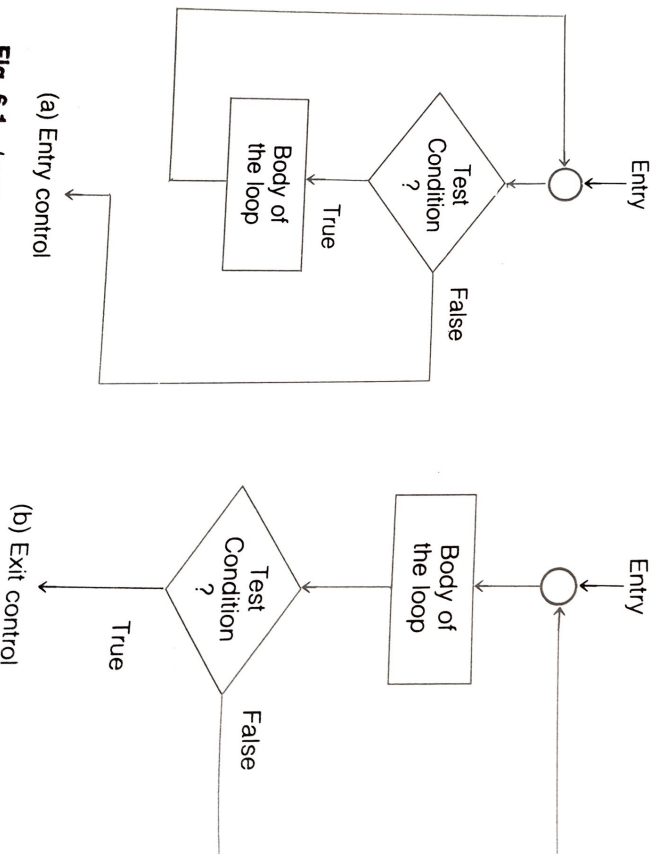


Fig. 6.1 Loop control structures

The test conditions should be carefully stated in order to perform the desired number of loop executions. It is assumed that the test condition will eventually transfer the control out of the loop. In case, due to some reason it does not do so, the control sets up an *infinite loop* and the body is executed over and over again.

A looping process, in general, would include the following four steps:

1. Setting and initialization of a counter.
2. Execution of the statements in the loop.
3. Test for a specified condition for execution of the loop.
4. Incrementing the counter.

The test may be either to determine whether the loop has been repeated the specified number of times or to determine whether a particular condition has been met.

The C language provides for three loop constructs for performing loop operations. They are:

1. The **while** statement.
2. The **do** statement.
3. The **for** statement.

We shall discuss the features and applications of each of these statements in this chapter.

6.2 THE WHILE STATEMENT

The simplest of all the looping structures in C is the **while** statement. We have used **while** in many of our earlier programs. The basic format of the **while** statement is

```
while (test condition)
{
    body of the loop
}
```

The **while** is an *entry-controlled* loop statement. The *test-condition* is evaluated and if the condition is true, then the body of the loop is executed. After execution of the body, the test-condition is once again evaluated and if it is true, the body is executed once again. This process of repeated execution of the body continues until the test-condition finally becomes *false* and the control is transferred out of the loop. On exit, the program continues with the statement immediately after the body of the loop.

The body of the loop may have one or more statements. The braces are needed only if the body contains two or more statements. However, it is a good practice to use braces even if the body has only one statement.

We can rewrite the program loop discussed in Section 6.1 as follows:

```
-----
sum = 0;
n = 1;
while(n <= 10)
{
    sum = sum + n * n;
    n = n + 1;
}
printf("sum = %d\n", sum);
-----
```

The body of the loop is executed 10 times for $n = 1, 2, \dots, 10$ each time adding the square of the value of n , which is incremented inside the loop. The test condition may also be written as $n < 11$; the result would be the same.

Another example of **while** statement which uses the keyboard input is shown below:

```

-----
-----
character = ' '
while (character != 'Y')
    character = getchar();
xxxxxxx;
-----
-----

```

First the **character** is initialized to ' '. The **while** statement then begins by testing whether **character** is not equal to Y. Since the **character** was initialized to ' ', the test is true and the loop statement

character = getchar();

is executed. Each time a letter is keyed in, the test is carried out and the loop statement is executed until the letter Y is pressed. When Y is pressed, the condition becomes false because **character** equals Y, and the loop terminates, thus transferring the control to the statement **xxxxxxx;**.

Example 6.1

A program to evaluate the equation

$$y = x^n$$

when n is a non-negative integer, is given in Fig. 6.2.

The variable y is initialized to 1 and then multiplied by x , n times using the **while** loop. The loop control variable, **count** is initialized outside the loop and incremented inside the loop. When the value of **count** becomes greater than n , the control exits the loop.

6.3 THE DO STATEMENT

The **while** loop construct that we have discussed in the previous section makes a test of condition *before* the loop is executed. Therefore, the body of the loop may not be executed at all if the condition is not satisfied at the very first attempt. On some occasions it might be necessary to execute the body of the loop before the test is performed. Such situations can be handled with the help of the **do** statement. This takes the form:

```

do
{
    body of the loop
}
while (test-condition);

```

On reaching the **do** statement, the program proceeds to evaluate the body of the loop first. At the end of the loop, the *test-condition* in the **while** statement is evaluated. If the condition is

Program

```

/*****
/*                               EXAMPLE OF while STATEMENT                               */
/*****
main( )
{
    int count, n;
    float x, y;

    printf("Enter the values of x and n : ");
    scanf("%f %d", &x, &n);
    y = 1.0;
    count = 1;

    /* LOOP BEGINS */
    while ( count <= n)
    {
        y = y*x;
        count++;
    }

    /* END OF LOOP */
    printf(" x = %f; n = %d; x to power n = %f n",x,n,y);
}

```

Output

```

Enter the values of x and n : 2.5  4
x = 2.500000; n = 4; x to power n = 39.062500

Enter the values of x and n : 0.5  4
x = 0.500000; n = 4; x to power n = 0.062500

```

Fig. 6.2 Program to compute x to the power n using **while** loop

true, the program continues to evaluate the body of the *loop* once again. This process continues as long as the *condition* is true. When the condition becomes false, the loop will be terminated and the control goes to the statement that appears immediately after the **while** statement.

Since the *test-condition* is evaluated at the bottom of the loop, the **do...while** construct provides an *exit-controlled* loop and therefore the body of the loop is *always executed at least once*.

A simple example of a **do...while** loop is:

```

-----
do
{
    printf("Input a number\n");
    number = getnum( );
}
while (number > 0);
-----

```

This segment of a program reads a number from the keyboard until a zero or a negative number is keyed in.

The test conditions may have compound relations as well. For instance, the statement

```
while (number > 0 && number < 100);
```

in the above example would cause the loop to be executed as long as the number keyed in lies between 0 and 100.

Consider another example:

```
-----
l = 1;
sum = 0;
do
{
    sum = sum + l;
    l = l+2;
}
while(sum < 40 || l < 10);
printf("%d %d\n", l, sum);
-----
```

The loop will be executed as long as one of the two relations is true.

Example 6.2

A program to print the multiplication table from 1×1 to 12×10 as shown below is given in Fig. 6.3.

1	2	3	4		10
2	4	6	8		20
3	6	9	12		30
4					40
.					.
.					.
.					.
12					120

This program contains two **do....while** loops in nested form. The outer loop is controlled by the variable **row** and executed 12 times. The inner loop is controlled by the variable **column** and is executed 10 times, each time the outer loop is executed. That is, the inner loop is executed a total of 120 times, each time printing a value in the table.

Notice that the **printf** of the inner loop does not contain any new line character (**\n**). This allows the printing of all row values in one line. The empty **printf** in the outer loop initiates a new line to print the next row.

6.4 THE FOR STATEMENT

Simple 'for' Loops

The **for** loop is another *entry-controlled* loop that provides a more concise loop control structure. The general form of the **for** loop is

```
for ( initialization ; test-condition ; increment)
{
    body of the loop
}
```

Program

```

/*****
/*      PRINTING OF MULTIPLICATION TABLE      */
*****/

#define COLMAX 10
#define ROWMAX 12

main( )
{
    int row,column, y;
    row = 1;
    printf("      MULTIPLICATION TABLE      \n");
    printf("      \n");
    /*.....OUTER LOOP BEGINS.....*/
    do
    {
        column = 1;
        /*.....INNER LOOP BEGINS.....*/
        do
        {
            y = row * column;
            printf("%4d", y);
            column = column + 1;
        }
        while (column <= COLMAX);
        /*... INNER LOOP ENDS ...*/
        printf("\n");
        row = row + 1;
    }
    while (row <= ROWMAX);
    /*..... OUTER LOOP ENDS .....*/
    printf("      \n");
}

```

Output

MULTIPLICATION TABLE

1	2	3	4	5	6	7	8	9	10
2	4	6	8	10	12	14	16	18	20
3	6	9	12	15	18	21	24	27	30
4	8	12	16	20	24	28	32	36	40
5	10	15	20	25	30	35	40	45	50
6	12	18	24	30	36	42	48	54	60
7	14	21	28	35	42	49	56	63	70
8	16	24	32	40	48	56	64	72	80
9	18	27	36	45	54	63	72	81	90
10	20	30	40	50	60	70	80	90	100
11	22	33	44	55	66	77	88	99	110
12	24	36	48	60	72	84	96	108	120

Fig. 6.3 Printing of a multiplication table using **do...while** loop

The execution of the `for` statement is as follows:

1. *Initialization* of the *control variables* is done first, using assignment statements such as `i = 1` and `count = 0`. The variables `i` and `count` are known as loop-control variables.
 2. The value of the control variable is tested using the *test-condition*. The test-condition is a relational expression, such as `i < 10` that determines when the loop will exit. If the condition is *true*, the body of the loop is executed; otherwise the loop is terminated and the execution continues with the statement that immediately follows the loop.
 3. When the body of the loop is executed, the control is transferred back to the `for` statement after evaluating the last statement in the loop. Now, the control variable is *incremented* using an assignment statement such as `i = i+1` and the new value of the control variable is again tested to see whether it satisfies the loop condition. If the condition is satisfied, the body of the loop is again executed. This process continues till the value of the control variable fails to satisfy the test-condition.
- Consider the following segment of a program:

```
for ( x = 0 ; x <= 9 ; x = x+1 )
{
    printf("%d", x);
}
printf("\n");
```

This `for` loop is executed 10 times and prints the digits 0 to 9 in one line. The three sections enclosed within parentheses must be separated by semicolons. Note that there is no semicolon at the end of the *increment* section, `x = x+1`.

The `for` statement allows for *negative increments*. For example, the loop discussed above can be written as follows:

```
for ( x = 9 ; x >= 0 ; x = x-1 )
    printf("%d", x);
printf("\n");
```

This loop is also executed 10 times, but the output would be from 9 to 0 instead of 0 to 9. Note that braces are optional when the body of the loop contains only one statement.

Since the conditional test is always performed at the beginning of the loop, the body of the loop may not be executed at all, if the condition fails at the start. For example,

```
for ( x = 9 ; x < 9 ; x = x-1 )
    printf("%d", x);
```

will never be executed because the test condition fails at the very beginning itself.

Let us again consider the problem of sum of squares of integers discussed in section 6.1. This problem can be coded using the `for` statement as follows:

```
-----
-----
sum = 0;
for ( n = 1 ; n <= 10 ; n = n+1 )
{
    sum = sum + n*n;
}
printf("sum = %d\n", sum);
-----
-----
```

The body of the loop

sum = sum + n*n;

is executed 10 times for $n = 1, 2, \dots, 10$ each time incrementing the **sum** by the square of the value of **n**.

One of the important points about the **for** loop is that all the three actions, namely *initialization*, *testing*, and *incrementing*, are placed in the **for** statement itself, thus making them visible to the programmers and users, in one place. The **for** statement and its equivalent of **while** and **do** statements are shown in Table 6.1.

Table 6.1 Comparison of the Three Loops

for	while	do
for ($n=1; n \leq 10; ++n$) { ----- ----- }	$n = 1;$ while ($n \leq 10$) { ----- ----- $n = n+1;$ }	$n = 1;$ do { ----- ----- $n = n+1;$ } while ($n \leq 10$);

Example 6.3

The program in Fig. 6.4 uses a **for** loop to print the "Powers of 2" table for the power 0 to 20, both positive and negative.

The program evaluates the value

$$p = 2^n$$

successively by multiplying 2 by itself n times.

$$q = 2^{-n} = \frac{1}{p}$$

Note that we have declared **p** as a **long int** and **q** as a **double**.

Additional Features of for Loop

The **for** loop in C has several capabilities that are not found in other loop constructs. For example, more than one variable can be initialized at a time in the **for** statement. The statements

p = 1;
for ($n=0; n < 17; ++n$)

can be rewritten as

for ($p=1, n=0; n < 17; ++n$)

```

Program
/*****
USE OF for LOOP
*****/
/*
main( )
{
    long int p;
    int n;
    double q;
    printf("-----\n");
    printf(" 2 to power n      n      2 to power -n\n");
    printf("-----\n");
    p = 1;
    for (n = 0; n < 21; ++n) /* LOOP BEGINS */
    {
        if (n == 0)
            p = 1;
        else
            p = p * 2;
        q = 1.0/(double)p;
        printf("%10ld %10d %20.12lf\n", p, n, q);
    } /* LOOP ENDS */
    printf("-----\n");
}

```

Output

2 to power n	n	2 to power -n
1	0	1.000000000000
2	1	0.500000000000
4	2	0.250000000000
8	3	0.125000000000
16	4	0.062500000000
32	5	0.031250000000
64	6	0.015625000000
128	7	0.007812500000
256	8	0.003906250000
512	9	0.001953125000
1024	10	0.000976562500
2048	11	0.000488281250
4096	12	0.000244140625
8192	13	0.000122070313
16384	14	0.000061035156
32768	15	0.000030517578
65536	16	0.000015258789
131072	17	0.000007629395
262144	18	0.000003814697
524288	19	0.000001907349
1048576	20	0.000000953674

Fig. 6.4 Program to print 'Power of 2' table using **for** loop

Notice that the initialization section has two parts $p = 1$ and $n = 1$ separated by a *comma*. Like the initialization section, the increment section may also have more than one part. For example, the loop

```
for (n=1, m=50; n<=m; n=n+1, m=m-1)
{
    p = m/n;
    printf("%d %d %d\n", n, m, p);
}
```

is perfectly valid. The multiple arguments in the increment section are separated by *commas*. The third feature is that the test-condition may have any compound relation and the testing need not be limited only to the loop control variable. Consider the example below:

```
sum = 0;
for (i = 1; i < 20 && sum < 100; ++i)
{
    sum = sum+i;
    printf("%d %d\n", sum);
}
```

The loop uses a compound test condition with the control variable i and external variable sum . The loop is executed as long as both the conditions $i < 20$ and $sum < 100$ are true. The sum is evaluated inside the loop.

It is also permissible to use expressions in the assignment statements of initialization and increment sections. For example, a statement of the type

```
for (x = (m+n)/2; x > 0; x = x/2)
```

is perfectly valid.

Another unique aspect of **for** loop is that one or more sections can be omitted, if necessary. Consider the following statements:

```
-----
-----
m = 5;
for ( ; m != 100 ; )
{
    printf("%d\n", m);
    m = m+5;
}
-----
-----
```

Both the initialization and increment sections are omitted in the **for** statement. The initialization has been done before the **for** statement and the control variable is incremented inside the loop. In such cases, the sections are left blank. However, the semicolons separating the sections must remain. If the test-condition is not present, the **for** statement sets up an infinite loop. Such loops can be broken using **break** or **goto** statements in the loop.

We can set up *time delay loops* using the *null* statement as follows:

```
for (j = 1000; j > 0; j = j-1)
;
```

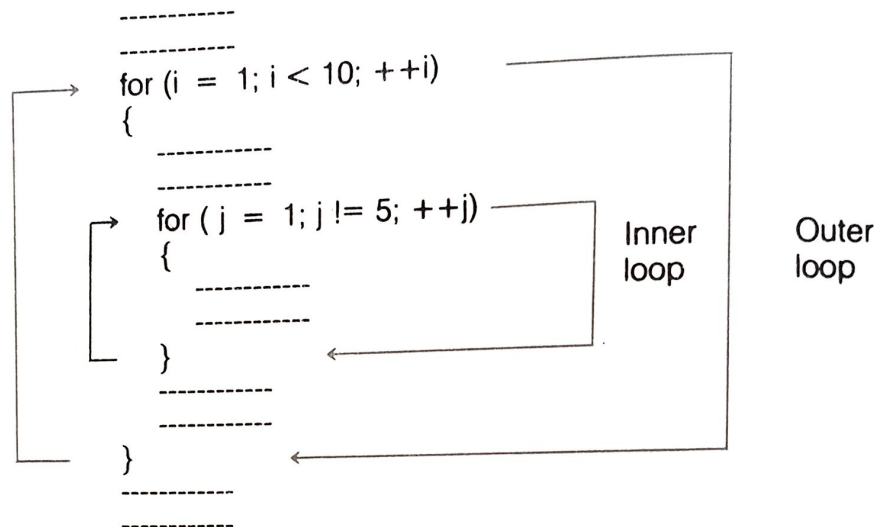
This loop is executed 1000 times without producing any output; it simply causes a time delay. Notice that the body of the loop contains only a semicolon, known as a *null* statement. This can also be written as

`for (j=1000; j > 0; j = j-1);`

This implies that the C compiler will not give an error message if we place a semicolon by mistake at the end of a `for` statement. The semicolon will be considered as a *null statement* and the program may produce some nonsense.

Nesting of for Loops

Nesting of loops, that is, one `for` statement within another `for` statement, is allowed in C. For example, two loops can be nested as follows:



The nesting may continue upto 15 levels in ANSI C; many compilers allow more. The loops should be properly indented so as to enable the reader to easily determine which statements are contained within each `for` statement.

The program to print the multiplication table discussed in Example 6.2 can be written more concisely using nested `for` statements as follows:

```
-----  
-----  
for (row = 1; row <= ROWMAX ; ++row)  
{  
    for (column = 1; column <= COLMAX ; ++column)  
    {  
        y = row * column;  
        printf("%4d", y);  
    }  
    printf("\n");  
}  
-----  
-----
```

The outer loop controls the rows while the inner loop controls the columns.

Example 6.4

A class of n students take an annual examination in m subjects. A program to read the marks obtained by each student in various subjects and to compute and print the total marks obtained by each of them is given in Fig. 6.5.

Program

```

/*****
/*          ILLUSTRATION OF NESTED LOOPS          */
*****/

#define FIRST      360
#define SECOND     240

main( )
{
    int n, m, i, j,
        roll-number, marks, total;
    printf("Enter number of students and subjects\n");
    scanf("%d %d", &n, &m);
    printf("\n");
    for (i = 1; i <= n; ++i)
    {
        printf("Enter roll-number : ");
        scanf("%d", &roll-number);
        total = 0;
        printf("\nEnter marks of %d subjects for ROLL NO %d\n", m, roll-number);
        for (j = 1; j <= m; j++)
        {
            scanf("%d", &marks);
            total = total + marks;
        }
        printf("TOTAL MARKS = %d ", total);
        if (total >= FIRST)
            printf("( First Division )\n\n");
        else if (total >= SECOND)
            printf("( Second Division )\n\n");
        else
            printf("( *** F A I L *** )\n\n");
    }
}

```

Output

```

Enter number of students and subjects
3 6
Enter roll-number : 8701
Enter marks of 6 subjects for ROLL NO 8701
81 75 83 45 61 59
TOTAL MARKS = 404 ( First Division )
Enter roll-number : 8702
Enter marks of 6 subjects for ROLL NO 8702
51 49 55 47 65 41
TOTAL MARKS = 308 ( Second Division )
Enter roll-number : 8704
Enter marks of 6 subjects for ROLL NO 8704
40 19 31 47 39 25
TOTAL MARKS = 201 ( *** F A I L *** )

```

Fig. 6.5 Illustration of nested *for* loops

The program uses two **for** loops, one for controlling the number of students and the other for controlling the number of subjects. Since both the number of students and the number of subjects are requested by the program, the program may be used for a class of any size and any number of subjects.

The outer loop includes three parts: (1) reading of roll-numbers of students, one after another, (2) inner loop, where the marks are read and totalled for each student, and (3) printing of total marks and declaration of grades.

6.5 JUMPS IN LOOPS

Loops perform a set of operations repeatedly until the control variable fails to satisfy the test-condition. The number of times a loop is repeated is decided in advance and the test condition is written to achieve this. Sometimes, when executing a loop it becomes desirable to skip a part of the loop or to leave the loop as soon as a certain condition occurs. For example, consider the case of searching for a particular name in a list containing, say, 100 names. A program loop written for reading and testing the names a 100 times must be terminated as soon as the desired name is found. C permits a jump from one statement to another within a loop as well as a jump out of a loop.

Jumping Out of a Loop

An early exit from a loop can be accomplished by using the **break** statement or the **goto** statement. We have already seen the use of the **break** in the **switch** statement and the **goto** in the **if...else** construct. These statements can also be used within **while**, **do**, or **for** loops. They are illustrated in Fig. 6.6 and Fig. 6.7.

When the **break** statement is encountered inside a loop, the loop is immediately exited and the program continues with the statement immediately following the loop. When the loops are nested, the **break** would only exit from the loop containing it. That is, the **break** will exit only a single loop.

Since a **goto** statement can transfer the control to any place in a program, it is useful to provide branching within a loop. Another important use of **goto** is to exit from deeply nested loops when an error occurs. A simple **break** statement would not work here.

Example 6.5

The program in Fig. 6.8 illustrates the use of the **break** statement in a C program.

The program reads a list of positive values and calculates their average. The **for** loop is written to read 1000 values. However, if we want the program to calculate the average of any set of values less than 1000, then we must enter a negative number after the last value in the list, to mark the end of input.

Each value, when it is read, is tested to see whether it is a positive number or not. If it is positive, the value is added to the **sum**; otherwise, the loop terminates. On exit, the average of the values read is calculated and the results are printed out.

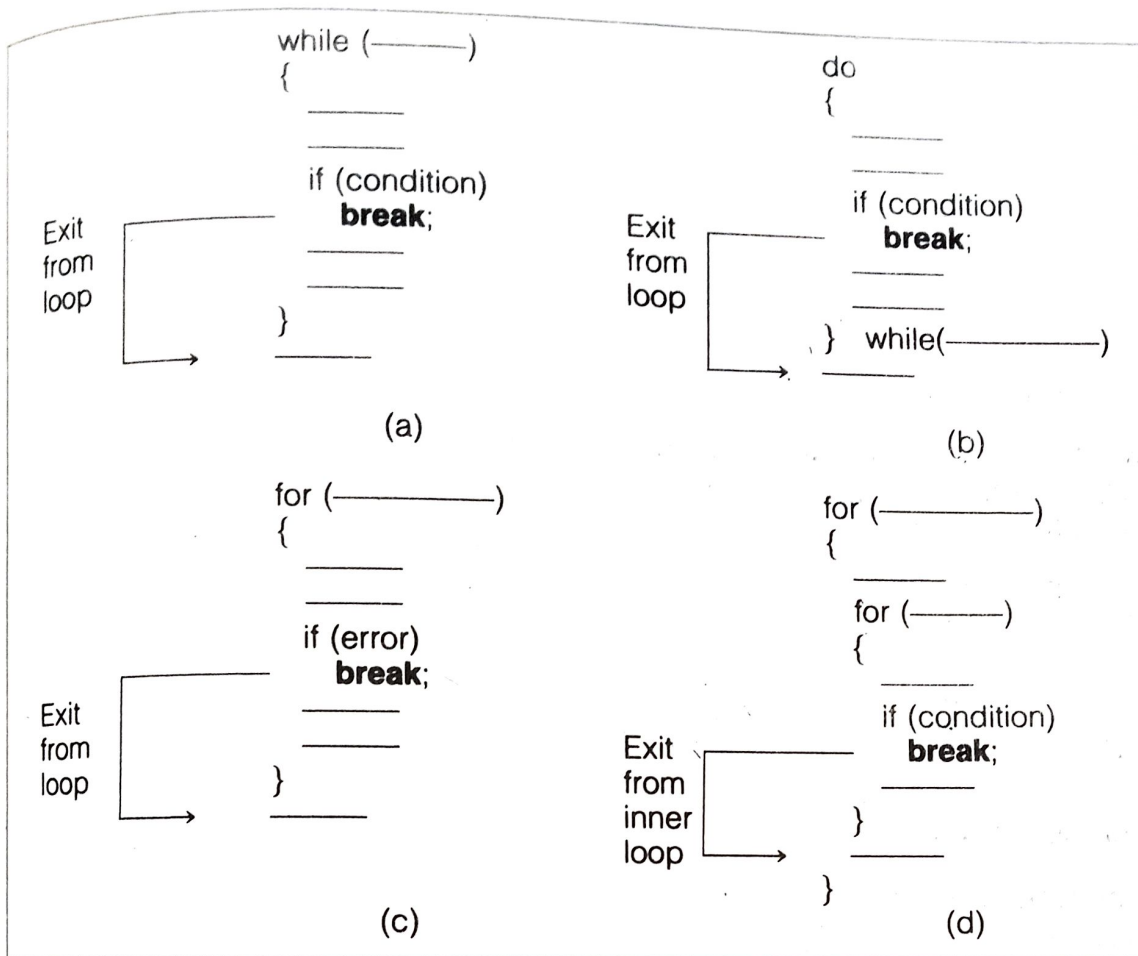


Fig. 6.6 Exiting a loop with **break** statement

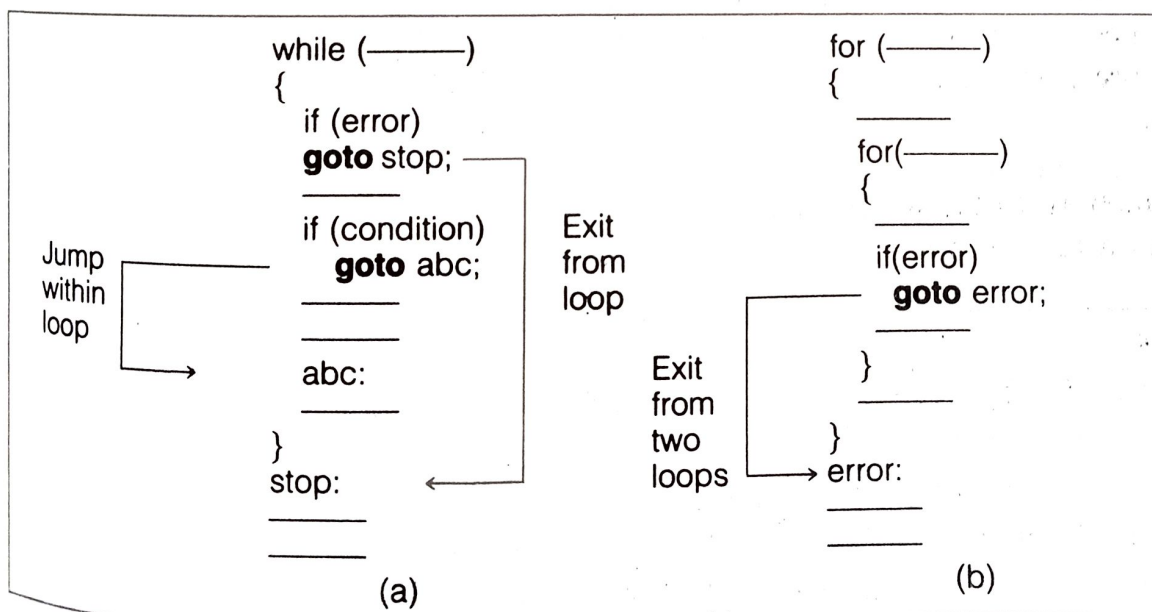


Fig. 6.7 Jumping within and exiting from the loops with **goto** statement

Program

```

/*****
 *
 *      USE OF break IN A PROGRAM
 *
 *****/
main( )
{
    int m;
    float x, sum, average;
    printf("This program computes the average of a
           set of numbers\n");
    printf("Enter values one after another\n");
    printf("Enter a NEGATIVE number at the end.\n\n");
    sum = 0;
    for (m = 1 ; m <= 1000 ; ++m)
    {
        scanf("%f", &x);
        if (x < 0)
            break;
        sum += x ;
    }
    average = sum/(float)(m-1);
    printf("\n");
    printf("Number of values = %d\n", m-1);
    printf("Sum              = %f\n", sum);
    printf("Average           = %f\n", average);
}

```

Output

```

This program computes the average of a set of numbers
Enter values one after another
Enter a NEGATIVE number at the end.
21 23 24 22 26 22 -1
Number of values = 6
Sum              = 138.000000
Average          = 23.000000

```

Fig. 6.8 Use of **break** in a program

Example 6.6

A program to evaluate the series

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \dots + x^n$$

for $-1 < x < 1$ to evaluate to 0.01 per cent accuracy is given in Fig. 6.9. The **goto** statement is used to exit the loop on achieving the desired accuracy.

We have used the **for** statement to perform the repeated addition of each of the terms in the series. Since it is an infinite series, the evaluation of the function is terminated when the term

x^* reaches the desired accuracy. The value of n that decides the number of loop operations is not known and therefore we have decided arbitrarily a value of 100, which may or may not result in the desired level of accuracy.

```

Program
/*****
EXAMPLE OF exit WITH goto STATEMENT
*****/
#define LOOP 100
#define ACCURACY 0.0001
main()
{
    int n;
    float x, term, sum;
    printf("Input value of x : ");
    scanf("%f", &x);
    sum = 0;
    for (term = 1, n = 1; n <= LOOP; ++n)
    {
        sum += term;
        if (term <= ACCURACY)
            goto output; /* EXIT FROM THE LOOP */
        term *= x;
    }
    printf("\nFINAL VALUE OF N IS NOT SUFFICIENT\n");
    printf("TO ACHIEVE DESIRED ACCURACY\n");
    goto end;
output:
    printf("\nEXIT FROM LOOP\n");
    printf("Sum = %f; No.of terms = %d\n", sum, n);
    end:
}

Output
Input value of x : .21
EXIT FROM LOOP
Sum = 1.265800; No.of terms = 7

Input value of x : .75
EXIT FROM LOOP
Sum = 3.999774; No.of terms = 34

Input value of x : .99
FINAL VALUE OF N IS NOT SUFFICIENT
TO ACHIEVE DESIRED ACCURACY

```

Fig. 6.9 Use of **goto** to exit from a loop

The test of accuracy is made using an **if** statement and the **goto** statement exits the loop as soon as the accuracy condition is satisfied. If the number of loop repetitions is not large enough to produce the desired accuracy, the program prints an appropriate message.

Note that the **break** statement is not very convenient to use here. Both the normal exit and

the **break** exit will transfer the control to the same statement that appears next to the loop. But, in the present problem, the *normal exit* prints the message

"FINAL VALUE OF N IS NOT SUFFICIENT
TO ACHIEVE DESIRED ACCURACY"

and the *forced exit* prints the results of evaluation. Notice the use of a *null* statement at the end. This is necessary because a program should not end with a label.

Skipping a part of a Loop

During the loop operations, it may be necessary to skip a part of the body of the loop under certain conditions. For example, in processing of applications for some job, we might like to exclude the processing of data of applicants belonging to a certain category. On reading the category code of an applicant, a test is made to see whether his application should be considered or not. If it is not to be considered, the part of the program loop that processes the application details is skipped and the execution continues with the next loop operation.

Like the **break** statement, C supports another similar statement called the **continue** statement. However, unlike the **break** which causes the loop to be terminated, the **continue**, as the name implies, causes the loop to be *continued* with the *next iteration* after *skipping* any statements in between. The **continue** statement tells the compiler, "SKIP THE FOLLOWING STATEMENTS AND CONTINUE WITH THE NEXT ITERATION". The format of the **continue** statement is simply

continue;

The use of the **continue** statement in loops is illustrated in Fig. 6.10. In **while** and **do** loops, **continue** causes the control to go directly to the *test-condition* and then to **continue** the iteration process. In the case of **for** loop, the *increment* section of the loop is executed before the *test-condition* is evaluated.

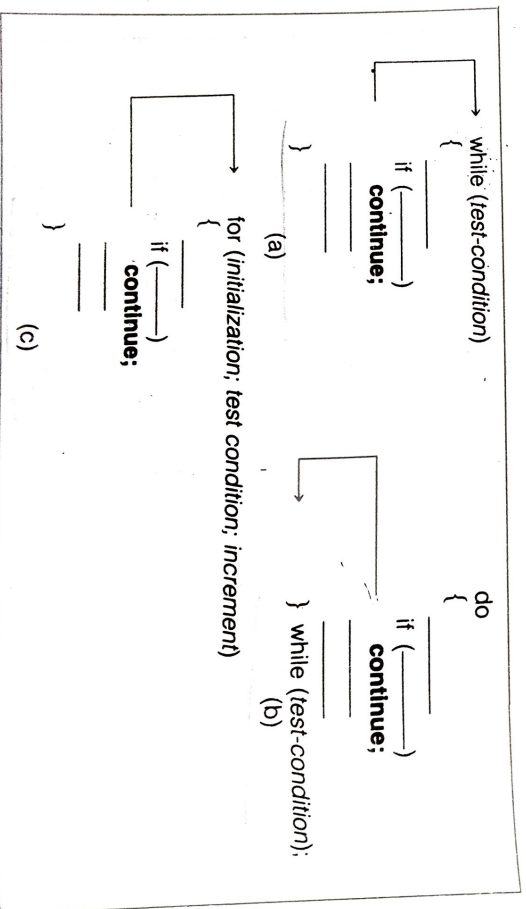


Fig. 6.10 Bypassing and continuing in loops

Example 6.7

The program in Fig. 6.11 illustrates the use of *continue* statement.

The program evaluates the square root of a series of numbers and prints the results. The process stops when the number 9999 is typed in.

In case, the series contains any negative numbers, the process of evaluation of square root would be bypassed for such numbers because the square root of a negative number is not defined. The *continue* statement is used to achieve this. The program also prints a message saying that the number is negative and keeps an account of negative numbers.

The final output includes the number of positive values evaluated and the number of negative items encountered.

```

Program
/*****
/***** USE OF continue STATEMENT *****/
/***** */
#include <math.h>

main()
{
    int count, negative;
    double number, sqroot;
    printf("Enter 9999 to STOP\n");
    count = 0;
    negative = 0;
    while (count <= 100)
    {
        printf("Enter a number : ");
        scanf("%f", &number);
        if (number == 9999)
            break;
        if (number < 0)
        {
            printf("Number is negative\n\n");
            negative++;
            continue; /* SKIP REST OF THE LOOP */
        }
        sqroot = sqrt(number);
        printf("Number      = %f\nSquare root = %f\n\n",
            number, sqroot);
        count++;
    }
    printf("Number of items done = %d\n", count);
    printf("\n\nNegative items      = %d\n", negative);
    printf("END OF DATA\n");
}

Output
Enter 9999 to STOP
Enter a number : 25.0

```

```

Number      = 25.000000
Square root = 5.000000

Enter a number : 40.5
Number      = 40.500000
Square root = 6.363961

Enter a number : -9
Number is negative

Enter a number : 16
Number      = 16.000000
Square root = 4.000000

Enter a number : -14.75
Number is negative

Enter a number : 80
Number      = 80.000000
Square root = 8.944272

Enter a number : 9999

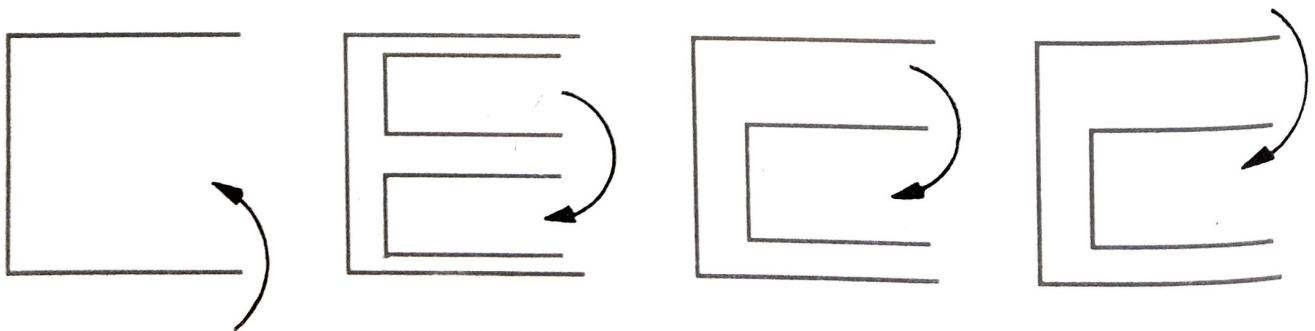
Number of items done = 4
Negative items       = 2
END OF DATA

```

Fig. 6.11 Use of **continue** statement

Avoiding goto

As mentioned earlier, it is a good practice to avoid using **gotos**. There are many reasons for this. When **goto** is used, many compilers generate a less efficient code. In addition, using many of them makes a program logic complicated and renders the program unreadable. It is possible to avoid using **goto** by careful program design. In case any **goto** is absolutely necessary, it should be documented. The following **goto** jumps would cause problems and therefore must be avoided:



Concise Test Expressions

We often use test expressions in the if, for, while and do statements that are evaluated and compared with zero for making branching decisions. Since every integer expression has a true/false value, we need not make explicit comparisons with zero. For instance, the expression x is true whenever x is not zero, and false when x is zero. Applying $!$ operator, we can write concise test expressions without using any relational operators.

if ($expression == 0$)

is equivalent to

if ($! expression$)

Similarly,

if ($expression != 0$)

is equivalent to

if ($expression$)

Example

If $(m \% 5 == 0 \ \&\& \ n \% 5 == 0)$ is same as if $(! (m \% 5) \ \&\& \ ! (n \% 5))$

CASE STUDIES

1. Table of Binomial Coefficients

Problem: Binomial coefficients are used in the study of binomial distributions and reliability of multicomponent redundant systems. It is given by

$$B(m, x) = \binom{m}{x} = \frac{m!}{x! (m-x)!}, m \geq x$$

A table of binomial coefficients is required to determine the binomial coefficient for any set of m and x .

Problem Analysis: The binomial coefficient can be recursively calculated as follows:

$$B(m, 0) = 1$$

$$B(m, x) = B(m, x-1) \left[\frac{m-x+1}{x} \right], \quad x = 1, 2, 3, \dots, m$$

Further,

$$B(0, 0) = 1$$

That is, the binomial coefficient is one when either x is zero or m is zero. The program in Fig. 6.12 prints the table of binomial coefficients for $m = 10$. The program employs one **do** loop and one **while** loop.

Program

```

/*****
*
* EVALUATION OF BINOMIAL COEFFICIENTS
*
*****/

#define MAX 10

main( )
{
    int m, x, binom;
    printf(" m x");
    for (m = 0; m <= 10; ++m)
        printf("%4d", m);
    printf("\n ----- \n");

    m = 0;
    do
    {
        printf("%2d ", m);
        x = 0; binom = 1;
        while (x <= m)
        {
            if(m == 0 || x == 0)
                printf("%4d", binom);
            else
            {
                binom = binom * (m - x + 1)/x;
                printf("%4d", binom);
            }
            x = x + 1;
        }
        printf("\n");
        m = m + 1;
    }
    while (m <= MAX);
    printf(" ----- \n");
}

```

Output

m x	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	1	1									
2	1	2	1								
3	1	3	3	1							
4	1	4	6	4	1						
5	1	5	10	10	5	1					
6	1	6	15	20	15	6	1				
7	1	7	21	35	35	21	7	1			
8	1	8	28	56	70	56	28	8	1		
9	1	9	36	84	126	126	84	36	9	1	
10	1	10	45	120	210	252	210	120	45	10	1

Fig. 6.12 Program to print binomial coefficient table

2. Histogram

Problem: In an organization, the employees are grouped according to their basic pay for the purpose of certain perks. The pay-range and the number of employees in each group are as follows:

Group	Pay-Range	Number of Employees
1	750 – 1500	12
2	1501 – 3000	23
3	3001 – 4500	35
4	4501 – 6000	20
5	above 6000	11

Draw a histogram to highlight the group sizes.

Problem Analysis: Given the size of groups, it is required to draw bars representing the sizes of various groups. For each bar, its group number and size are to be written.

Program in Fig. 6.13 reads the number of employees belonging to each group and draws a histogram. The program uses four for loops and two if....else statements.

Program

```

/*****
/*          PROGRAM TO DRAW HISTOGRAM          */
*****/

#define      N      5

main( )
{
    int value[N];
    int i, j, n, x;
    for (n=0; n < N; ++n)
    {
        printf("Enter employees in Group - %d : ",n+1);
        scanf("%d", &x);
        value[n] = x;
        printf("%d\n", value[n]);
    }
    printf("\n");
    printf("      |\n");
    for (n = 0 ; n < N ; ++n)
    {
        for(i = 1 ; i <= 3 ; i++)
        {
            if ( i == 2)
                printf("Group-%1d      |",n+1);
            else
                printf("      |");
            for (j = 1 ; j <= value[n]; ++j)
                printf("***");
            if (i == 2)
                printf("(%d)\n", value[n]);
            else

```

```
printf("\n");
}
printf("\n");
```

```
Enter employees in Group - 1 : 12
12
Enter employees in Group - 2 : 23
23
Enter employees in Group - 3 : 35
35
Enter employees in Group - 4 : 20
20
Enter Employees in Group - 5 : 11
11
```

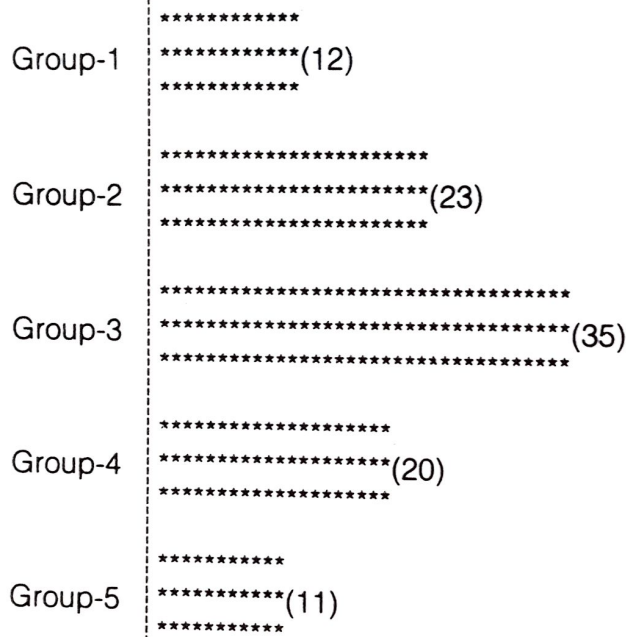


Fig. 6.13 *Program to draw a histogram*

3. Minimum Cost

Problem: The cost of operation of a unit consists of two components C_1 and C_2 which can be expressed as functions of a parameter p as follows:

$$\begin{aligned} C_1 &= 30 - 8p \\ C_2 &= 10 + p^2 \end{aligned}$$

The parameter p ranges from 0 to 10. Determine the value of p with an accuracy of ± 0.1 where the cost of operation would be minimum.

Problem Analysis:

$$\text{Total cost} = C_1 + C_2 = 40 - 8p + p^2$$

The cost is 40 when $p = 0$, and 33 when $p = 1$ and 60 when $p = 10$. The cost, therefore, decreases first and then increases. The program in Fig. 6.14 evaluates the cost at successive

intervals of p (in steps of 0.1) and stops when the cost begins to increase. The program employs **break** and **continue** statements to exit the loop.

Program

```

/*****
*                               *
*      PROBLEM OF MINIMUM COST      *
*                               *
*****/

main( )
{
    float p, cost, p1, cost1;
    for (p = 0; p <= 10; p = p + 0.1)
    {
        cost = 40 - 8 * p + p * p;
        if(p == 0)
        {
            cost1 = cost;
            continue;
        }
        if(cost >= cost1)
            break;
        cost1 = cost;
        p1 = p;
    }
    p = (p + p1)/2.0;
    cost = 40 - 8 * p + p * p;
    printf("\nMINIMUM COST = %.2f  AT p = %.1f\n", cost, p);
}

```

Output

MINIMUM COST = 24.00 AT p = 4.0

Fig. 6.14 Program of minimum cost problem

4. Plotting of Two Functions

Problem: We have two functions of the type

$$\begin{aligned}
 y_1 &= \exp(-ax) \\
 y_2 &= \exp(-ax^2/2)
 \end{aligned}$$

Plot the graphs of these functions for x varying from 0 to 5.0.

Problem Analysis: Initially when $x = 0$, $y_1 = y_2 = 1$ and the graphs start from the same point. The curves cross when they are again equal at $x = 2.0$. The program should have appropriate branch statements to print the graph points at the following three conditions:

1. $y_1 > y_2$
2. $y_1 < y_2$
3. $y_1 = y_2$

The functions y_1 and y_2 are normalized and converted to integers as follows:

$$\begin{aligned}
 y_1 &= 50 \exp(-ax) + 0.5 \\
 y_2 &= 50 \exp(-ax^2/2) + 0.5
 \end{aligned}$$

The program in Fig. 6.15 plots these two functions simultaneously. (0 for y1, * for y2, and # for the common point).

Program

```

/*****
/*      \          PLOTTING OF TWO FUNCTIONS          */
*****/

#include    <math.h>

main( )
{
    int i;
    float a, x, y1, y2;
    a = 0.4;
    printf("          Y _____> \n");
    printf(" 0 _____ \n");

    for (x = 0; x < 5; x = x+0.25)
    { /* BEGINNING OF FOR LOOP */
        /* .....Evaluation of functions..... */
        y1 = (int) ( 50 * exp( -a * x ) + 0.5 );
        y2 = (int) (50 * exp( -a * x * x/2 ) + 0.5 );

        /* .....Plotting when y1 = y2 ..... */
        if (y1 == y2)
        {
            if (x == 2.5)
                printf(" X |");
            else
                printf(" |");
            for (i = 1; i <= y1 - 1; ++i)
                printf(" ");
            printf("#\n");
            continue;
        }

        /* .....Plotting when y1 > y2 ..... */
        if (y1 > y2)
        {
            if (x == 2.5)
                printf(" X |");
            else
                printf(" |");
            for (i = 1; i <= y2 - 1; ++i)
                printf(" ");
            printf(" *");
            for (i = 1; i <= (y1 - y2 - 1); ++i)
                printf("-");
            printf("0\n");
            continue;
        }
    }
}

```

```

    }
    /* .....Plotting when y2 > y1 ..... */
    if (x == 2.5)
        printf(" X |");
    else
        printf(" |");
    for (i = 1; i <= (y1 - 1); ++i)
        printf(" ");
    printf("0");
    for (i = 1; i <= (y2 - y1 - 1); ++i)
        printf("-");
    printf("*\n");
} /* .....END OF FOR LOOP ..... */
printf(" |\n");
}

```

Output

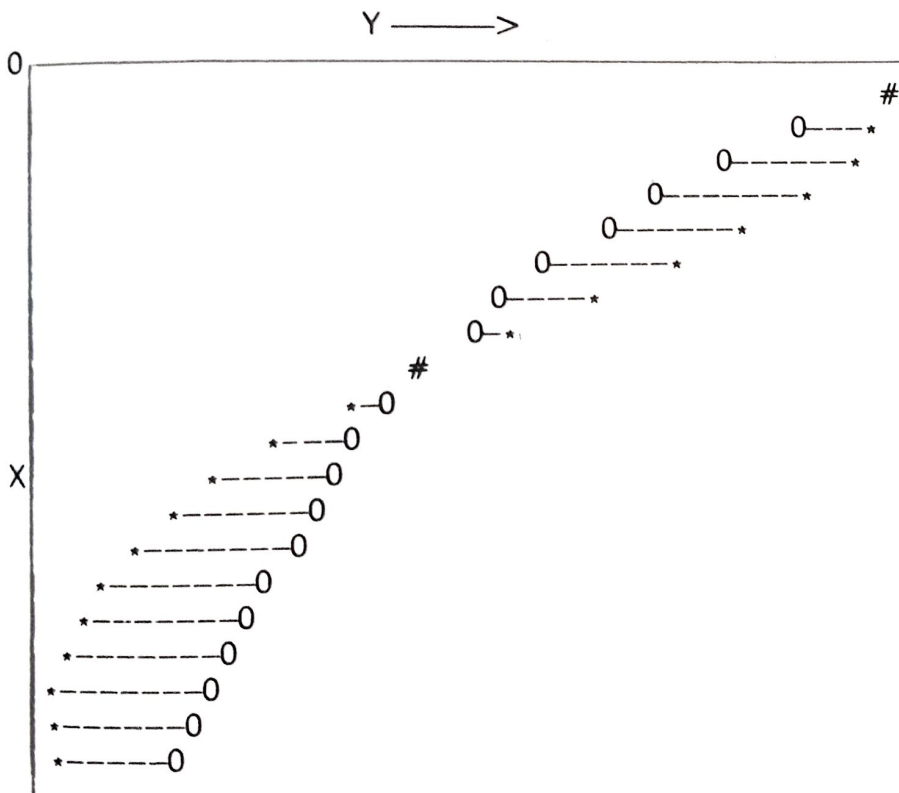


Fig. 6.15 Plotting of two functions

REVIEW QUESTIONS AND EXERCISES

- 61 Compare in terms of their functions, the following pairs of statements:
- while and do...while.
 - while and for.
 - break and goto.
 - break and continue.
 - continue and goto.

6.2 Analyze each of the program segments that follow and determine how many times the body of each loop will be executed.

```
(a) x = 5;
    y = 50;
    while ( x <= y)
    {
        x = y/x;
        _____
        _____
    }
```

```
(b) m = 1;
    do
    {
        _____
        _____
        m = m+2;
    }
    while (m < 10);
```

```
(c) int i;
    for (i = 0; i <= 5; i = i+2/3)
    {
        _____
        _____
        _____
    }
```

```
(d) int m = 10;
    int n = 7;
    while ( m % n >= 0)
    {
        _____
        m = m + 1;
        n = n + 2;
        _____
    }
```

6.3 Find errors, if any, in each of the following looping segments. Assume that all the variables have been declared and assigned values.

```
(a) while (count != 10);
    {
        count = 1;
        sum = sum + x;
        count = count + 1;
    }
```

```
(b) name = 0;
    do { name = name + 1;
        printf("My name is John\n"); }
    while (name = 1)
```

```
(c) do;
    total = total + value;
    scanf("%f", &value);
    while(value != 999);
```

```
(d) for (x = 1, x > 10; x = x + 1)
{
    _____
    _____
    _____
}
```

```
(e) m = 1;
    n = 0;
    for ( ; m+n < 19; ++n);
    printf("Hello\n");
    m = m+10
```

```
(f) for (p = 10; p > 0;)
    p = p - 1;
    printf("%f", p);
```

6.4 What is a null statement? Explain its usefulness.

6.5 Given a number, write a program using **while** loop to reverse the digits of the number. For example, the number

12345

should be written as

54321

(Hint: Use modulus operator to extract the last digit and the integer division by 10 to get the $n-1$ digit number from the n digit number.)

6.6 The factorial of an integer m is the product of consecutive integers from 1 to m . That is,
factorial $m = m! = m \times (m-1) \times \dots \times 1$.

Write a program that computes and prints a table of factorials for any given m .

6.7 Write a program to compute the sum of the digits of a given integer number.

6.8 The numbers in the sequence

1 1 2 3 5 8 13 21

are called Fibonacci numbers. Write a program using a **do....while** loop to calculate and print the first m Fibonacci numbers.

(Hint: After the first two numbers in the series, each number is the sum of the two preceding numbers.)

6.9 Rewrite the program of the Example 6.1 using the **for** statement.

6.10 Write a program to evaluate the following investment equation

$$V = P(1+r)^n$$

and print the tables which would give the value of V for various combination of the following values of P , r , and n .

P : 1000, 2000, 3000,, 10,000

r : 0.10, 0.11, 0.12,, 0.20

n : 1, 2, 3,, 10

(Hint: P is the principal amount and V is the value of money at the end of n years. This equation can be recursively written as

$$V = P(1+r)$$

$$P = V$$

That is, the value of money at the end of first year becomes the principal amount for the next year and so on.)

6.11 Write programs to print the following outputs using for loops.

(a) 1
2 2
3 3 3
4 4 4 4
5 5 5 5 5

(b) * * * * *
* * * * *
* * * * *
* * * * *
* * * * *

(c) 1
2 2
3 3 3
4 4 4 4
5 5 5 5 5

6.12 Write a program to read the age of 100 persons and count the number of persons in the age group 50 to 60. Use for and continue statements.

6.13 Rewrite the program of case study 6.4 (plotting of two curves) using else...if constructs instead of continue statements.

6.14 Write a program to print a table of values of the function

$$y = \exp(-x)$$

for x varying from 0.0 to 10.0 in steps of 0.10. The table should appear as follows:

Table for $Y = \text{EXP}(-X)$

x	0.1	0.2	0.3		0.9
0.0					
1.0					
2.0					
3.0					
.					
.					
.					
9.0					

6.15 Write a program that will read a positive integer and determine and print its binary equivalent.

(Hint: The bits of the binary representation of an integer can be generated by repeatedly dividing the number and the successive quotients by 2 and saving the remainder, which is either 0 or 1, after each division.)